

# Cluster Set up and Sharding

## Part1 Self deployed MongoDB Cluster

### Key Cluster Concepts

Before we start off, I wanted to get some concepts across

**What is a shard** – A shard is a storage part of MongoDB sharded cluster. It works by instead of storing all data on one server, MongoDB splits the data across multiple shards. Each shard stores a part of the collection. In relation to this project, four shards were used, northRS, southRS, eastRS, and westRS. Each shard is used to represent one data centre region which contains three MongoDB nodes.

**What is a config server** – config server is a server that stores metadata about the sharded cluster, this metadata includes info about shards, chunks, shard keys and where data is stored. In relation to this project, three config server nodes were used and organised as the cfgRS replica set. Finally, having three config server is important because one server alone would be a single point of failure.

**What is an interface in the cluster** – an interface in the cluster is the mongos router which clients connect to instead of connecting directly to the shards. Mongos does not store app data; it uses metadata from the config servers to route queries and writes to the correct shard. Now, if there was only one mongos process and it failed clients would lose access to the cluster even if the shards were still running.

**What is a chunk** - A chunk is a range of sharded data. When used in MongoDB it splits a sharded collection into chunks and distributes those chunks across shards. If a chunk then becomes too large, MongoDB can split it into smaller chunks. If one shard has too many chunks, MongoDB can move chunks to other shards to balance the cluster.

**How to change chunk size** – in order to change the chunk size in a config database, the following command would be needed to change the chunk size `db.settings.updateOne({_id: "chunksize" }, { $set: { value: 32 } }, { upsert: true })`, this would set the chunk size to 32MB

**What is migration** - Migration is the process where MongoDB moves chunks from one shard to another. This happens when MongoDB tries to balance data across shards. Finally, migration helps prevent one shard from storing too much data compared to others.

**What is splitting** – Splitting is the process that happens when a chunk grows too large. The way it works is MongoDB divides that chunk into smaller chunks so that the data can be distributed more evenly across the cluster. For example, if many employee records fall into the same department range, MongoDB may split that chunk into smaller ranges.

**What is a shard key / justification** - A shard key is the field that MongoDB uses to decide how data is distributed across the shards in a sharded cluster. So when a document is inserted, MongoDB looks at the shard key value and uses it to decide which shard should store that document.

This is important because it affects how evenly the data is spread across the cluster. A poor shard key can cause too much data to go to one shard, which creates an unbalanced cluster. A good shard key helps spread the data more evenly and improves query performance.

For this project, the shard key used was { department: 1, employee\_id: 1 }.

I chose this shard key because the project queries often use the department field to group or analyse employees. The employee\_id field was also included to make the shard key more specific. This helps avoid too many employees being grouped under only one department value.

Using both fields together helps MongoDB distribute the employee records more evenly across the four shards while still supporting department-based queries. This makes the shard key suitable for this dataset and the required queries.

## Q1 Cluster Setup

### Project Structure

Before starting the cluster, I created a main project folder which I named mongoClusterProject and inside this folder I created separate folders for scripts, data and logs. The scripts folders store the batch files and java script files used to start and config the cluster. Data folder stores a separate data directory for each MongoDB node. Lastly the logs folder stores log files created by each MongoDB process

This is required as the project requires a script based setup for a simulated MongoDB cluster or one physical machine. The spec also says the cluster is simulated locally and should be built using batch or java script files

## Config Server Setup

Starting, the first script created was 01\_start\_config\_servers.bat , this script starts three config server nodes, cfg0, cfg1 and cfg2. Each config server uses a different port and its own data folder. Each of these config servers are needed as they store the metadata for the sharded cluster. This data includes info about shards, chunks and where data is stored.

In the case for starting the config server nodes, the command used was `--configsvr` as the nodes are not normal data nodes, their job is to support the sharded cluster by storing cluster metadata.

This relates to the marking point asking what a config server is, how many nodes it should have and what data stores. The project spec also states that the config server should have three nodes organized as replica set

| This PC > Windows-SSD (C:) > mongoClusterProject > |                  |             |
|----------------------------------------------------|------------------|-------------|
| Name                                               | Date modified    | Type        |
| data                                               | 20/05/2026 13:46 | File folder |
| logs                                               | 20/05/2026 13:44 | File folder |
| scripts                                            | 20/05/2026 14:19 | File folder |

| This PC > Windows-SSD (C:) > mongoClusterProject > scripts |                  |                 |
|------------------------------------------------------------|------------------|-----------------|
| Name                                                       | Date modified    | Type            |
| 01_start_config_servers.bat                                | 20/05/2026 14:14 | Windows Batch F |

|             |
|-------------|
| Select cfg0 |
| Select cfg1 |
| Select cfg2 |

```
echo Starting config server cfg0 on port 26050...
start "cfg0" mongod --configsvr --replSet cfgRS --port 26050 --dbpath C:\mongoClusterProject\data\cfg0 --logpath C:\mongoClusterProject\logs\cfg0.log --logappend

echo Starting config server cfg1 on port 26051...
start "cfg1" mongod --configsvr --replSet cfgRS --port 26051 --dbpath C:\mongoClusterProject\data\cfg1 --logpath C:\mongoClusterProject\logs\cfg1.log --logappend

echo Starting config server cfg2 on port 26052...
start "cfg2" mongod --configsvr --replSet cfgRS --port 26052 --dbpath C:\mongoClusterProject\data\cfg2 --logpath C:\mongoClusterProject\logs\cfg2.log --logappend

echo.
echo Config server nodes started.
pause
```

## Initialising the Config Server Replica Set

After creating and starting the config servers, I created 02\_init\_config\_rs.js, this java script file initialises the config server replica set called cfgRS which contains the localhost 26050, 26051 and 26052. The command includes configsvr: true, this tells MongoDB that this replica set is being used as the config server replica set for the sharded cluster. After running the script, I verified the setup using rs.status(), which showed one config server node as primary and the other as secondary

```
\scripts>mongosh --port 26050 C:\mongoClusterProject\scripts\02_init_config_rs.js
```

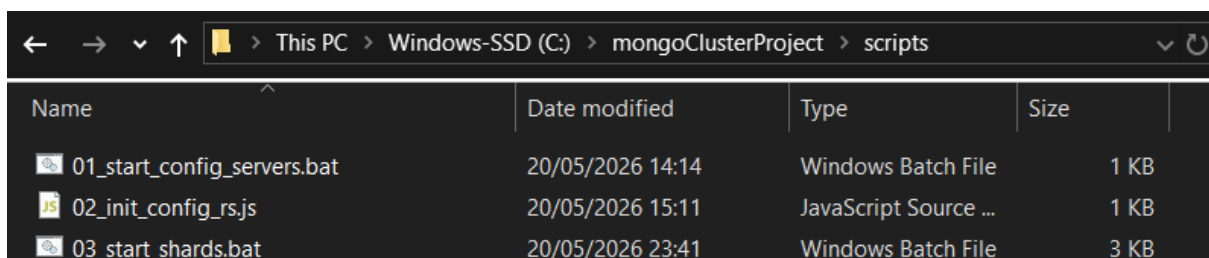
```
{
  members: [
    {
      _id: 0,
      name: 'localhost:26050',
      health: 1,
      state: 1,
      stateStr: 'PRIMARY',
      uptime: 3109,
      optime: { ts: Timestamp({ t: 1779286335, i: 1 }),
      optimeDate: ISODate('2026-05-20T14:12:15.000Z'),
      optimeWritten: { ts: Timestamp({ t: 1779286335, i: 1 }),
      optimeWrittenDate: ISODate('2026-05-20T14:12:15.000Z'),
      lastAppliedWallTime: ISODate('2026-05-20T14:12:15.000Z'),
      lastDurableWallTime: ISODate('2026-05-20T14:12:15.000Z'),
      lastWrittenWallTime: ISODate('2026-05-20T14:12:15.000Z'),
      syncSourceHost: '',
      syncSourceId: -1,
      infoMessage: '',
      electionTime: Timestamp({ t: 1779286178, i: 1 }),
      electionDate: ISODate('2026-05-20T14:09:38.000Z'),
      configVersion: 1,
      configTerm: 1,
      self: true,
      lastHeartbeatMessage: ''
    },
    {
      _id: 1,
      name: 'localhost:26051',
      health: 1,
      state: 2,
      stateStr: 'SECONDARY',
```

```
rs.initiate({
  _id: "cfgRS",
  configsvr: true,
  members: [
    { _id: 0, host: "localhost:26050" },
    { _id: 1, host: "localhost:26051" },
    { _id: 2, host: "localhost:26052" }
  ]
});
```

## Sharded Server Setup

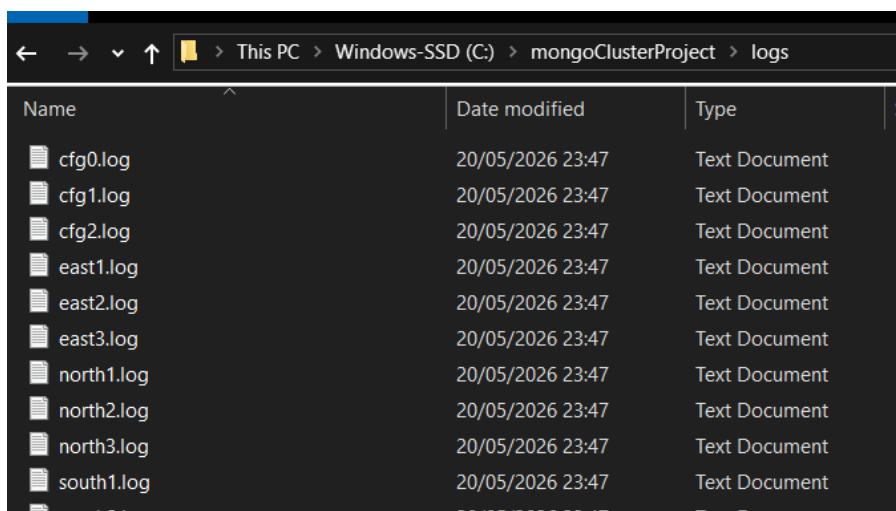
The next script created was 03\_start\_shards.bat, this script starts the 12 shard nodes required for the cluster, the project requires 12 nodes distributed across four regions, north, south, east and west. Each region acts as one shard and each shard is organized as a replica set. This matches the project requirements that each data centre represents a shard and each shard stores part of the application data

The shard replica sets are, northRS, southRS, eastRS and westRS, each shard contains three MongoDB nodes. For example, the north shard contains north1, 2 and 3. Each shard node is started with the --shardsvr option because these nodes store the actual app data in the sharded cluster.



A screenshot of a Windows File Explorer window showing the contents of the 'scripts' directory. The address bar indicates the path is 'This PC > Windows-SSD (C:) > mongoClusterProject > scripts'. The table below lists the files in this directory.

| Name                        | Date modified    | Type                  | Size |
|-----------------------------|------------------|-----------------------|------|
| 01_start_config_servers.bat | 20/05/2026 14:14 | Windows Batch File    | 1 KB |
| 02_init_config_rs.js        | 20/05/2026 15:11 | JavaScript Source ... | 1 KB |
| 03_start_shards.bat         | 20/05/2026 23:41 | Windows Batch File    | 3 KB |



A screenshot of a Windows File Explorer window showing the contents of the 'logs' directory. The address bar indicates the path is 'This PC > Windows-SSD (C:) > mongoClusterProject > logs'. The table below lists the log files in this directory.

| Name       | Date modified    | Type          | S |
|------------|------------------|---------------|---|
| cfg0.log   | 20/05/2026 23:47 | Text Document |   |
| cfg1.log   | 20/05/2026 23:47 | Text Document |   |
| cfg2.log   | 20/05/2026 23:47 | Text Document |   |
| east1.log  | 20/05/2026 23:47 | Text Document |   |
| east2.log  | 20/05/2026 23:47 | Text Document |   |
| east3.log  | 20/05/2026 23:47 | Text Document |   |
| north1.log | 20/05/2026 23:47 | Text Document |   |
| north2.log | 20/05/2026 23:47 | Text Document |   |
| north3.log | 20/05/2026 23:47 | Text Document |   |
| south1.log | 20/05/2026 23:47 | Text Document |   |
| south2.log | 20/05/2026 23:47 | Text Document |   |
| south3.log | 20/05/2026 23:47 | Text Document |   |

```
echo Starting NORTH shard replica set nodes...
start "north1" mongod --shardsvr --replSet northRS --port 27101 --dbpath C:\mongoClusterProject\data\north1 --logpath C:\mongoClusterProject\logs\north1.log --logappend
start "north2" mongod --shardsvr --replSet northRS --port 27102 --dbpath C:\mongoClusterProject\data\north2 --logpath C:\mongoClusterProject\logs\north2.log --logappend
start "north3" mongod --shardsvr --replSet northRS --port 27103 --dbpath C:\mongoClusterProject\data\north3 --logpath C:\mongoClusterProject\logs\north3.log --logappend

echo Starting SOUTH shard replica set nodes...
start "south1" mongod --shardsvr --replSet southRS --port 27201 --dbpath C:\mongoClusterProject\data\south1 --logpath C:\mongoClusterProject\logs\south1.log --logappend
start "south2" mongod --shardsvr --replSet southRS --port 27202 --dbpath C:\mongoClusterProject\data\south2 --logpath C:\mongoClusterProject\logs\south2.log --logappend
start "south3" mongod --shardsvr --replSet southRS --port 27203 --dbpath C:\mongoClusterProject\data\south3 --logpath C:\mongoClusterProject\logs\south3.log --logappend

echo Starting EAST shard replica set nodes...
start "east1" mongod --shardsvr --replSet eastRS --port 27301 --dbpath C:\mongoClusterProject\data\east1 --logpath C:\mongoClusterProject\logs\east1.log --logappend
start "east2" mongod --shardsvr --replSet eastRS --port 27302 --dbpath C:\mongoClusterProject\data\east2 --logpath C:\mongoClusterProject\logs\east2.log --logappend
start "east3" mongod --shardsvr --replSet eastRS --port 27303 --dbpath C:\mongoClusterProject\data\east3 --logpath C:\mongoClusterProject\logs\east3.log --logappend

echo Starting WEST shard replica set nodes...
start "west1" mongod --shardsvr --replSet westRS --port 27401 --dbpath C:\mongoClusterProject\data\west1 --logpath C:\mongoClusterProject\logs\west1.log --logappend
start "west2" mongod --shardsvr --replSet westRS --port 27402 --dbpath C:\mongoClusterProject\data\west2 --logpath C:\mongoClusterProject\logs\west2.log --logappend
start "west3" mongod --shardsvr --replSet westRS --port 27403 --dbpath C:\mongoClusterProject\data\west3 --logpath C:\mongoClusterProject\logs\west3.log --logappend

echo.
echo All shard server nodes started.
pause
```

## Initialising the Shard Replica Sets

After starting the shard previously created, I created a new scrip called 04\_init\_shards.js which when initialised the four shard replica sets both northRS, southRS, eastRS and westRS. Each replica set contains three nodes. After running the script, each shard replica set elects on primary node and keeps the other two as secondary nodes

This set up allows replications inside each shard and if one node fails then the shards can still continue working because the other replica set members have copies of the data.

This step shows that each shard is properly configured and that the cluster is being build with replication inside each shard

| Name                        | Date modified    | Type                  | Size |
|-----------------------------|------------------|-----------------------|------|
| 01_start_config_servers.bat | 20/05/2026 14:14 | Windows Batch File    | 1 KB |
| 02_init_config_rs.js        | 20/05/2026 15:11 | JavaScript Source ... | 1 KB |
| 03_start_shards.bat         | 20/05/2026 23:41 | Windows Batch File    | 3 KB |
| 04_init_shards.js           | 21/05/2026 01:11 | JavaScript Source ... | 2 KB |

```
C:\Users\reyco>mongosh --nodb C:\mongoClusterProject\scripts\04_init_shards.js
Initialising northRS...
Initialising southRS...
Initialising eastRS...
Initialising westRS...
All shard replica sets have been initialised.
```

```
{
  "_id": 0,
  "name": "localhost:27101",
  "health": 1,
  "state": 1,
  "stateStr": "PRIMARY",
  "uptime": 5260,
  "optime": { "ts": Timestamp({ t: 1779322527, i: 1 }), t: Long('1') },
  "optimeDate": ISODate("2026-05-21T00:15:27.000Z"),
  "optimeWritten": { "ts": Timestamp({ t: 1779322527, i: 1 }), t: Long('1') },
  "optimeWrittenDate": ISODate("2026-05-21T00:15:27.000Z"),
  "lastAppliedWallTime": ISODate("2026-05-21T00:15:27.502Z"),
  "lastDurableWallTime": ISODate("2026-05-21T00:15:27.502Z"),
  "lastWrittenWallTime": ISODate("2026-05-21T00:15:27.502Z"),
  "syncSourceHost": "",
  "syncSourceId": -1,
  "infoMessage": "Could not find member to sync from",
  "electionTime": Timestamp({ t: 1779322457, i: 1 }),
  "electionDate": ISODate("2026-05-21T00:14:17.000Z"),
  "configVersion": 1,
  "configTerm": 1,
  "self": true,
  "lastHeartbeatMessage": ""
},
{
  "_id": 1,
  "name": "localhost:27102",
  "health": 1,
  "state": 2,
  "stateStr": "SECONDARY",
  "uptime": 83,
  "optime": { "ts": Timestamp({ t: 1779322527, i: 1 }), t: Long('1') },
  "optimeDurable": { "ts": Timestamp({ t: 1779322527, i: 1 }), t: Long('1') },
  "optimeWritten": { "ts": Timestamp({ t: 1779322527, i: 1 }), t: Long('1') },
  "optimeDate": ISODate("2026-05-21T00:15:27.000Z"),
  "optimeDurableDate": ISODate("2026-05-21T00:15:27.000Z"),
  "optimeWrittenDate": ISODate("2026-05-21T00:15:27.000Z"),
  "lastAppliedWallTime": ISODate("2026-05-21T00:15:27.502Z"),
  "lastWrittenWallTime": ISODate("2026-05-21T00:15:27.502Z"),
  "lastHeartbeat": ISODate("2026-05-21T00:15:29.467Z"),
  "lastHeartbeatRecv": ISODate("2026-05-21T00:15:30.461Z"),
  "pingMs": Long('0'),
  "lastHeartbeatMessage": "",
  "syncSourceHost": "localhost:27101",
  "syncSourceId": 0,
  "infoMessage": "",
  "configVersion": 1,
  "configTerm": 1
},
{
  "_id": 2,
  "name": "localhost:27103",
  "health": 1,
  "state": 2,
  "stateStr": "SECONDARY",
  "uptime": 83,
  "optime": { "ts": Timestamp({ t: 1779322527, i: 1 }), t: Long('1') },
  "optimeDurable": { "ts": Timestamp({ t: 1779322527, i: 1 }), t: Long('1') },
  "optimeWritten": { "ts": Timestamp({ t: 1779322527, i: 1 }), t: Long('1') },
  "optimeDate": ISODate("2026-05-21T00:15:27.000Z"),
  "optimeDurableDate": ISODate("2026-05-21T00:15:27.000Z"),
  "optimeWrittenDate": ISODate("2026-05-21T00:15:27.000Z"),
  "lastAppliedWallTime": ISODate("2026-05-21T00:15:27.502Z"),
  "lastWrittenWallTime": ISODate("2026-05-21T00:15:27.502Z"),
  "lastHeartbeat": ISODate("2026-05-21T00:15:29.467Z"),
  "lastHeartbeatRecv": ISODate("2026-05-21T00:15:30.461Z"),
  "pingMs": Long('0'),
  "lastHeartbeatMessage": "",
  "syncSourceHost": "localhost:27101",
  "syncSourceId": 0,
  "infoMessage": "",
  "configVersion": 1,
  "configTerm": 1
}
}
```

```
7
8 print("Initialising northRS...");
9 conn = new Mongo("localhost:27101");
10 db = conn.getDB("admin");
11
12 db.runCommand({
13   replSetInitiate: {
14     _id: "northRS",
15     members: [
16       { _id: 0, host: "localhost:27101" },
17       { _id: 1, host: "localhost:27102" },
18       { _id: 2, host: "localhost:27103" }
19     ]
20   }
21 });
22
23
24 print("Initialising southRS...");
25 conn = new Mongo("localhost:27201");
26 db = conn.getDB("admin");
27
28 db.runCommand({
29   replSetInitiate: {
30     _id: "southRS",
31     members: [
32       { _id: 0, host: "localhost:27201" },
33       { _id: 1, host: "localhost:27202" },
34       { _id: 2, host: "localhost:27203" }
35     ]
36   }
37 });
38
39
40 print("Initialising eastRS...");
41 conn = new Mongo("localhost:27301");
42 db = conn.getDB("admin");
43
44 db.runCommand({
45   replSetInitiate: {
46     _id: "eastRS",
47     members: [
48       { _id: 0, host: "localhost:27301" },
49       { _id: 1, host: "localhost:27302" },
50       { _id: 2, host: "localhost:27303" }
51     ]
52   }
53 });
54
55
56 print("Initialising westRS...");
57 conn = new Mongo("localhost:27401");
58 db = conn.getDB("admin");
59
60 db.runCommand({
61   replSetInitiate: {
62     _id: "westRS",
63     members: [
64       { _id: 0, host: "localhost:27401" },
65       { _id: 1, host: "localhost:27402" },
66       { _id: 2, host: "localhost:27403" }
67     ]
68   }
69 });
```

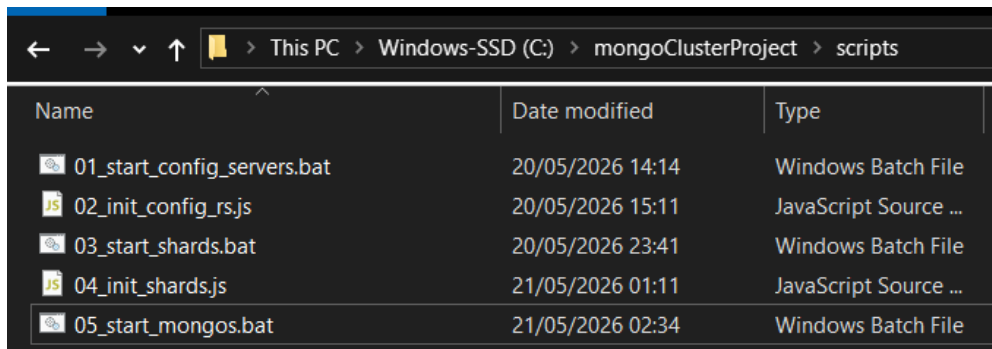
## Starting the Mongos Router

The next script is 05\_start\_mongos.bat this script will start the mongos router on port 27017

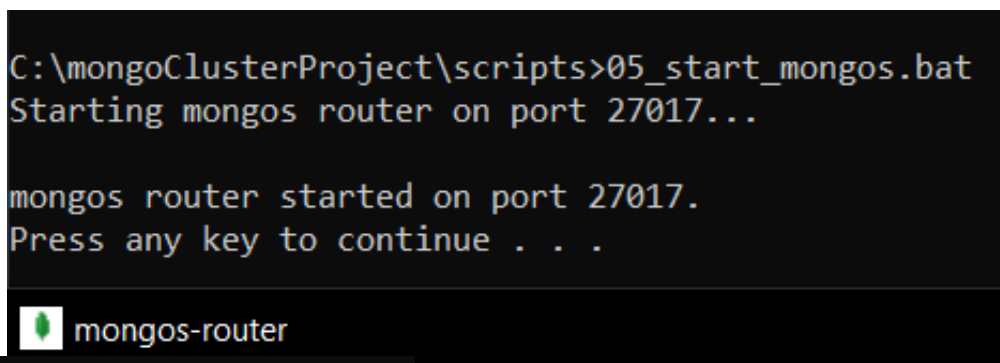
The mongos process is the interface between the client and the sharded cluster. Clients are not to connect directly to individual shards, instead they have to connect to mongos, and mongos decides where each request should go

The mongos router also connects to the config server replica set using cfgRS/localhost:26050,localhost:26051,localhost:26052. This allows mongos to read the cluster metadata and router queries to the correct shard.

This step relates directly to the spec marking point asking what an interface is in a cluster and whether it store data and what problems happen if there is only one interface. In this project the interface is the mongos router. It does not store app data, but it uses metadata from the config servers to route requests



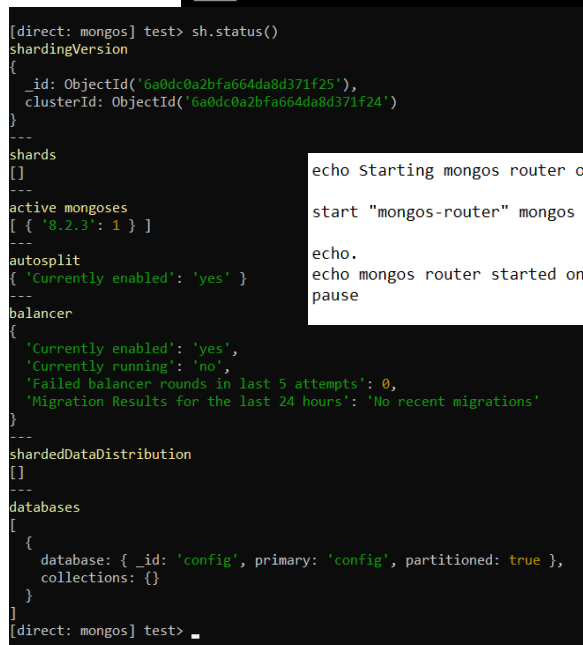
| Name                        | Date modified    | Type                  |
|-----------------------------|------------------|-----------------------|
| 01_start_config_servers.bat | 20/05/2026 14:14 | Windows Batch File    |
| 02_init_config_rs.js        | 20/05/2026 15:11 | JavaScript Source ... |
| 03_start_shards.bat         | 20/05/2026 23:41 | Windows Batch File    |
| 04_init_shards.js           | 21/05/2026 01:11 | JavaScript Source ... |
| 05_start_mongos.bat         | 21/05/2026 02:34 | Windows Batch File    |



```
C:\mongoClusterProject\scripts>05_start_mongos.bat
Starting mongos router on port 27017...

mongos router started on port 27017.
Press any key to continue . . .

 mongos-router
```



```
[direct: mongos] test> sh.status()
shardingVersion
{
  _id: ObjectId('6a0dc0a2bfa664da8d371f25'),
  clusterId: ObjectId('6a0dc0a2bfa664da8d371f24')
}
---
shards
[]
active mongoses
[ { '8.2.3': 1 } ]
autosplit
{ 'Currently enabled': 'yes' }
---
balancer
{
  'Currently enabled': 'yes',
  'Currently running': 'no',
  'Failed balancer rounds in last 5 attempts': 0,
  'Migration Results for the last 24 hours': 'No recent migrations'
}
---
shardedDataDistribution
[]
---
databases
[
  {
    database: { _id: 'config', primary: 'config', partitioned: true },
    collections: {}
  }
]
[direct: mongos] test>
```

```
echo Starting mongos router on port 27017...
```

```
start "mongos-router" mongos --configdb cfgRS/localhost:26050,localhost:26051,localhost:26052 --port 27017 --logpath C:\mongoCl
```

```
echo.
echo mongos router started on port 27017.
pause
```



## Adding the Shards to the Cluster

After the mongos router was started the next step was to add the four shard replica sets to cluster. In order to do this, I created a new scrip called 06\_add\_shards.js which when ran it add the following shard replica sets northRS, southRS, eastRS and westRS

Each shard represents one regional data centre in the cluster, the north, south, east and west shards which store the actual employee burnout application data once sharding is enabled, the shards were added using the sh.addShard() command. This command tells the mongos router that these replica sets are part of the sharded cluster. Finally, after the shards were added I used the sh.status() command to show and confirm that all shards were connected successfully.

This was an important step as it shows that a sharded cluster cannot distribute data until the shard replica sets are added to the cluster. The project also requires the cluster to be distributed across 4 regions with each data centre acting as one shard

| ← → ▾ ↑ 📁 > This PC > Windows-SSD (C:) > mongoClusterProject > scripts                                          |                  |                        |
|-----------------------------------------------------------------------------------------------------------------|------------------|------------------------|
| Name                                                                                                            | Date modified    | Type                   |
|  01_start_config_servers.bat | 20/05/2026 14:14 | Windows Batch File     |
|  02_init_config_rs.js        | 20/05/2026 15:11 | JavaScript Source File |
|  03_start_shards.bat         | 20/05/2026 23:41 | Windows Batch File     |
|  04_init_shards.js           | 21/05/2026 01:11 | JavaScript Source File |
|  05_start_mongos.bat         | 21/05/2026 02:34 | Windows Batch File     |
|  06_add_shards.js            | 21/05/2026 03:07 | JavaScript Source File |

```
C:\Users\reyco>mongosh --port 27017 C:\mongoClusterProject\scripts\06_add_shards.js
Adding northRS shard...
Adding southRS shard...
Adding eastRS shard...
Adding westRS shard...
All shards have been added to the cluster.
Cluster status:
```

```
[direct: mongos] test> sh.status()
shardingVersion
{
  _id: ObjectId('6a0dc0a2bfa664da8d371f25'),
  clusterId: ObjectId('6a0dc0a2bfa664da8d371f24')
}
---
shards
[
  {
    _id: 'eastRS',
    host: 'eastRS/localhost:27301,localhost:27302,localhost:27303',
    state: 1,
    topologyTime: Timestamp({ t: 1779329337, i: 79 }),
    replSetConfigVersion: Long('-1')
  },
  {
    _id: 'northRS',
    host: 'northRS/localhost:27101,localhost:27102,localhost:27103',
    state: 1,
    topologyTime: Timestamp({ t: 1779329335, i: 32 }),
    replSetConfigVersion: Long('-1')
  },
  {
    _id: 'southRS',
    host: 'southRS/localhost:27201,localhost:27202,localhost:27203',
    state: 1,
    topologyTime: Timestamp({ t: 1779329337, i: 24 }),
    replSetConfigVersion: Long('-1')
  },
  {
    _id: 'westRS',
    host: 'westRS/localhost:27401,localhost:27402,localhost:27403',
    state: 1,
    topologyTime: Timestamp({ t: 1779329338, i: 31 }),
    replSetConfigVersion: Long('-1')
  }
]
```

```
print("Adding northRS shard...");
sh.addShard("northRS/localhost:27101,localhost:27102,localhost:27103");

print("Adding southRS shard...");
sh.addShard("southRS/localhost:27201,localhost:27202,localhost:27203");

print("Adding eastRS shard...");
sh.addShard("eastRS/localhost:27301,localhost:27302,localhost:27303");

print("Adding westRS shard...");
sh.addShard("westRS/localhost:27401,localhost:27402,localhost:27403");

print("All shards have been added to the cluster.");

print("Cluster status:");
```

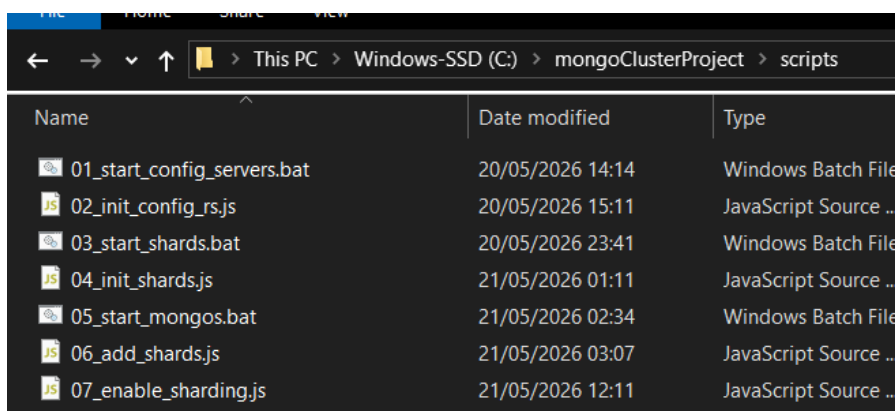


## Enabling Sharding and Preparing the Collection

The next script used was 07\_enable\_sharding.js which is a java script that enables sharding on the burnoutAnalysisDB database and creates the employee burnout collection. An index was also created on the shard key before the collection was sharded (shard key used: {department: 1, employee\_id: 1})

That shard key was chosen because the project queries frequently analyse the employees by department. The employee\_id field is also included to make the key more specific and help avoid too many records being grouped under the same value.

Finally, after the collection was sharded, I used the status command to confirm everything was recognised as a sharded collection.



| Name                        | Date modified    | Type                  |
|-----------------------------|------------------|-----------------------|
| 01_start_config_servers.bat | 20/05/2026 14:14 | Windows Batch File    |
| 02_init_config_rs.js        | 20/05/2026 15:11 | JavaScript Source ... |
| 03_start_shards.bat         | 20/05/2026 23:41 | Windows Batch File    |
| 04_init_shards.js           | 21/05/2026 01:11 | JavaScript Source ... |
| 05_start_mongos.bat         | 21/05/2026 02:34 | Windows Batch File    |
| 06_add_shards.js            | 21/05/2026 03:07 | JavaScript Source ... |
| 07_enable_sharding.js       | 21/05/2026 12:11 | JavaScript Source ... |

```
C:\Users\reyco>mongosh --port 27017 C:\mongoClusterProject\scripts\07_enable_sharding.js
Switching to burnoutAnalysisDB...
Creating employee_burnout collection...
Enabling sharding on burnoutAnalysisDB...
Creating index for shard key...
Sharding employee_burnout collection...
Sharding setup complete.
Cluster status:
```

```
[direct: mongos] test> sh.status()
shardingVersion
{
  _id: ObjectId('6a0dc0a2bfa664da8d371f25'),
  clusterId: ObjectId('6a0dc0a2bfa664da8d371f24')
}
---
shards
[
  {
    _id: 'eastRS',
    host: 'eastRS/localhost:27301,localhost:27302,localhost:27303',
    state: 1,
    topologyTime: Timestamp({ t: 1779329337, i: 79 }),
    replSetConfigVersion: Long('1')
  },
  {
    _id: 'northRS',
    host: 'northRS/localhost:27101,localhost:27102,localhost:27103',
    state: 1,
    topologyTime: Timestamp({ t: 1779329335, i: 32 }),
    replSetConfigVersion: Long('1')
  },
  {
    _id: 'southRS',
    host: 'southRS/localhost:27201,localhost:27202,localhost:27203',
    state: 1,
    topologyTime: Timestamp({ t: 1779329337, i: 24 }),
    replSetConfigVersion: Long('1')
  },
  {
    _id: 'westRS',
    host: 'westRS/localhost:27401,localhost:27402,localhost:27403',
    state: 1,
    topologyTime: Timestamp({ t: 1779329338, i: 31 }),
    replSetConfigVersion: Long('1')
  }
]
```

```
print("Switching to burnoutAnalysisDB...");
db = db.getSiblingDB("burnoutAnalysisDB");

print("Creating employee_burnout collection...");
db.createCollection("employee_burnout");

print("Enabling sharding on burnoutAnalysisDB...");
sh.enableSharding("burnoutAnalysisDB");

print("Creating index for shard key...");
db.employee_burnout.createIndex({ department: 1, employee_id: 1 });

print("Sharding employee_burnout collection...");
sh.shardCollection(
  "burnoutAnalysisDB.employee_burnout",
  { department: 1, employee_id: 1 }
);

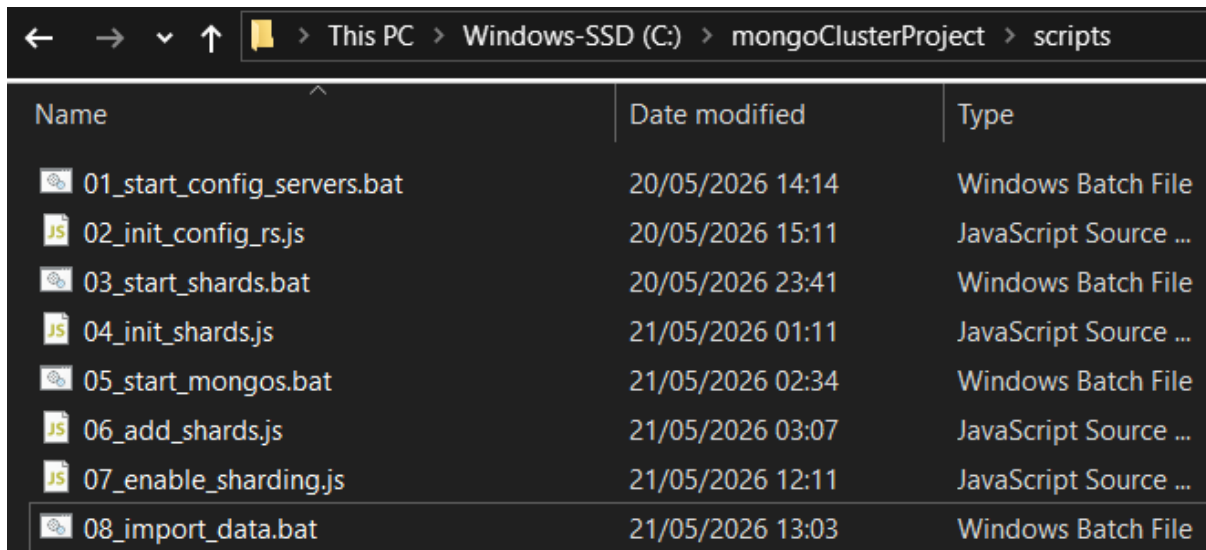
print("Sharding setup complete.");
print("Cluster status:");
```

## Importing the employee burnout dataset

08\_import\_data.bat is the next script created that I used. This scrip imports the employee burnout dataset into the employee\_burnout collection inside the burnoutAnalysisDB database.

The data was imported through the mongos router on port 27017. This is important as mongos acts as the entry point to the sharded cluster and routes the imported documents to the correct shards

Finally, after everything was imported, I checked the document count using the countDocuments() command in order to confirm everything was successfully inserted



| Name                                                                                                          | Date modified    | Type                  |
|---------------------------------------------------------------------------------------------------------------|------------------|-----------------------|
|  01_start_config_servers.bat | 20/05/2026 14:14 | Windows Batch File    |
|  02_init_config_rs.js        | 20/05/2026 15:11 | JavaScript Source ... |
|  03_start_shards.bat         | 20/05/2026 23:41 | Windows Batch File    |
|  04_init_shards.js          | 21/05/2026 01:11 | JavaScript Source ... |
|  05_start_mongos.bat       | 21/05/2026 02:34 | Windows Batch File    |
|  06_add_shards.js          | 21/05/2026 03:07 | JavaScript Source ... |
|  07_enable_sharding.js     | 21/05/2026 12:11 | JavaScript Source ... |
|  08_import_data.bat        | 21/05/2026 13:03 | Windows Batch File    |

```
C:\mongoClusterProject\scripts>08_import_data.bat
Importing employee burnout dataset into burnoutAnalysisDB.employee_burnout...
2026-05-21T13:06:53.198+0100 [#####] burnoutAnalysisDB.employee_burnout 1.22GB/1.22GB (100.0%)
2026-05-21T13:06:53.202+0100 849999 document(s) imported successfully. 0 document(s) failed to import.

Import complete.
Press any key to continue . . .
```

```
switched to db burnoutAnalysisDB
[direct: mongos] burnoutAnalysisDB> db.employee_burnout.countDocuments()

849999
```

```
echo Importing employee burnout dataset into burnoutAnalysisDB.employee_burnout...
```

```
mongoimport --host localhost --port 27017 ^
--db burnoutAnalysisDB ^
--collection employee_burnout ^
--file "C:\Users\reyco\Downloads\synthetic-employee-dataset.json" ^
--jsonArray
```

```
echo.
echo Import complete.
pause
```

## Checking the sharding status

09\_check\_sharding\_status.js is the next script, it is used to check whether the data had been imported correctly and whether the employee burnout collection was sharded.

Also, the script counted the number of documents, displaying the cluster status, it also shows collection statistics and listed the indexes on the collection

Finally, I used the status command to confirm that the cluster contained the four shard replica sets, north, south, east and west. It was also used to confirm that employee burnout collection belonged to the burnoutAnalysisDB and was configured as a sharded collection

| Name                                                                                                            | Date modified    | Type                  |
|-----------------------------------------------------------------------------------------------------------------|------------------|-----------------------|
|  01_start_config_servers.bat   | 20/05/2026 14:14 | Windows Batch File    |
|  02_init_config_rs.js          | 20/05/2026 15:11 | JavaScript Source ... |
|  03_start_shards.bat           | 20/05/2026 23:41 | Windows Batch File    |
|  04_init_shards.js             | 21/05/2026 01:11 | JavaScript Source ... |
|  05_start_mongos.bat          | 21/05/2026 02:34 | Windows Batch File    |
|  06_add_shards.js            | 21/05/2026 03:07 | JavaScript Source ... |
|  07_enable_sharding.js       | 21/05/2026 12:11 | JavaScript Source ... |
|  08_import_data.bat          | 21/05/2026 13:03 | Windows Batch File    |
|  09_check_sharding_status.js | 21/05/2026 13:39 | JavaScript Source ... |

```
C:\Users\reyco>mongosh --port 27017 C:\mongoClusterProject\scripts\09_check_sharding_status.js
Switching to burnoutAnalysisDB...
Counting documents in employee_burnout...
849999
Showing shard distribution and cluster status...
Showing collection statistics...
```

```
{
  'eviction walk pages queued that were dirty': 0,
  [direct: mongos] test> sh.status()
shardingVersion walk pages seen that were clean': 22214,
{
  'eviction walk pages seen that were dirty': 95,
  _id: ObjectId('6a0dc0a2bfa664da8d371f25'),
  clusterId: ObjectId('6a0dc0a2bfa664da8d371f24')': 0,
}
'eviction walk target pages histogram - 10-31': 4,
--- 'eviction walk target pages histogram - 128 and higher': 0,
shards'eviction walk target pages histogram - 32-63': 2,
[
  'eviction walk target pages histogram - 64-128': 11,
  {
    'eviction walk target pages reduced due to history store cache pressure': 0,
    _id: 'eastRS',ter blocked page eviction': 0,
    host: 'eastRS/localhost:27301,localhost:27302,localhost:27303',
    state: 1,y store table insert calls that returned restart': 0,
    topologyTime: Timestamp({ t: 1779329337, i: 79 })),
    replSetConfigVersion: Long('1')ssed': 0,
  }, 'history store table reads requiring squashed modifies': 0,
  {
    'history store table resolved updates without timestamps that lose their dur
    _id: 'northRS',e table truncation by rollback to stable to remove an unstable
    host: 'northRS/localhost:27101,localhost:27102,localhost:27103',e an update':
    state: 1,y store table truncation to remove all the keys of a btree': 0,
    topologyTime: Timestamp({ t: 1779329335, i: 32 })),ate': 0,
    replSetConfigVersion: Long('1')on to remove range of updates due to an update
  }, 'history store table truncation to remove range of updates due to key being
  _id: 'southRS',e table truncations that would have happened in non-dryrun mode
  host: 'southRS/localhost:27201,localhost:27202,localhost:27203',that would hav
  state: 1,y store table truncations to remove an update that would have happene
  topologyTime: Timestamp({ t: 1779329337, i: 24 })),fixed up by reinserting with
  replSetConfigVersion: Long('1')equiring squashed modifies': 0,
  }, 'in-memory page passed criteria to be split': 87,
  {
    'in-memory page splits': 29,
    _id: 'westRS',ge split blocked its eviction': 0,
    host: 'westRS/localhost:27401,localhost:27402,localhost:27403',
    state: 1,al pages split during eviction': 0,
    topologyTime: Timestamp({ t: 1779329338, i: 31 })),
    replSetConfigVersion: Long('1')y examining all entries on the root page': 0,
  }, 'modified pages evicted': 55,
  }, 'multi-block reconciliation blocked whilst checkpoint is running': 0,
}
```

```
print("Switching to burnoutAnalysisDB...");
db = db.getSiblingDB("burnoutAnalysisDB");

print("Counting documents in employee_burnout...");
printjson(db.employee_burnout.countDocuments());

print("Showing shard distribution and cluster status...");
sh.status();

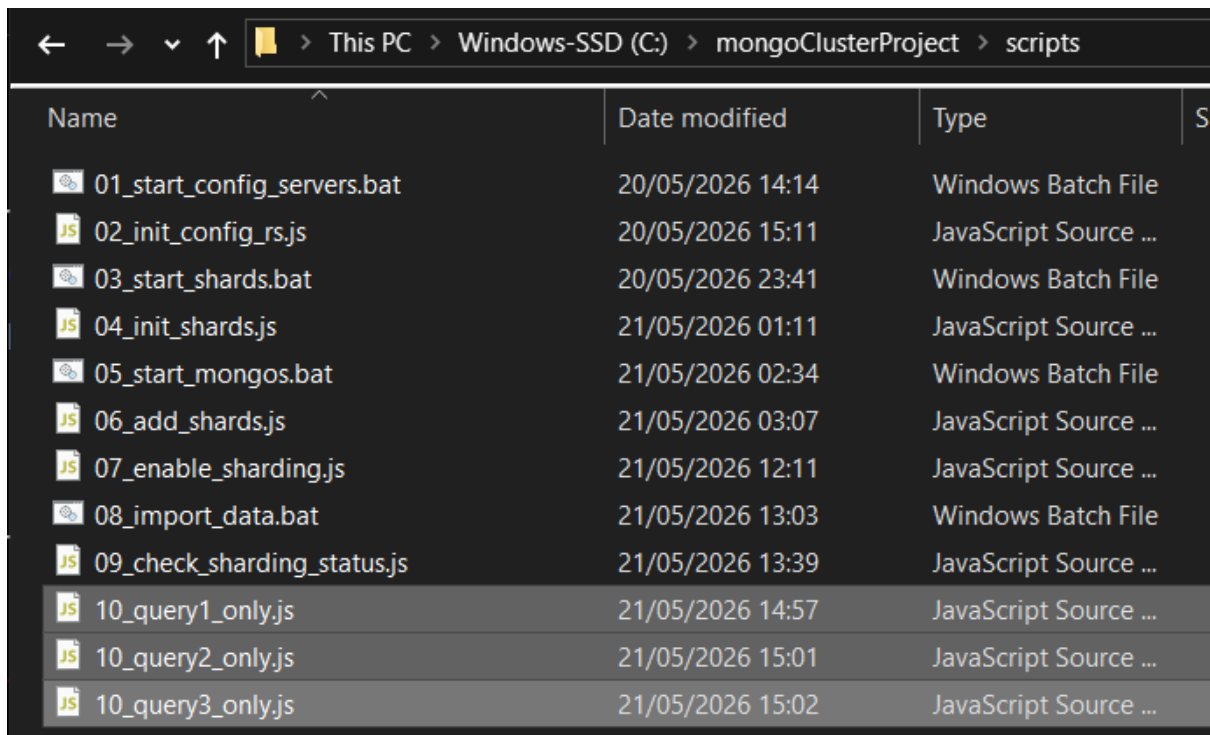
print("Showing collection statistics...");
printjson(db.employee_burnout.stats());

print("Checking shard key index...");
printjson(db.employee_burnout.getIndexes());
```

## Running the required Queries

Finally, after the cluster was created, sharding enabled and the employee burnout dataset was imported, the final step was to run the three required queries. These queries were executed through the mongos router on port 27017, rather than directly on an individual shard. This is necessary as mongos is the correct entry point for a sharded MongoDB cluster and it routes each query to the correct shards.

To keep everything clear, each query was placed in a separate java script file which made it easier to show each command and output for the this report and the screen recording.



| Name                                                                                                            | Date modified    | Type                  | S |
|-----------------------------------------------------------------------------------------------------------------|------------------|-----------------------|---|
|  01_start_config_servers.bat   | 20/05/2026 14:14 | Windows Batch File    |   |
|  02_init_config_rs.js          | 20/05/2026 15:11 | JavaScript Source ... |   |
|  03_start_shards.bat           | 20/05/2026 23:41 | Windows Batch File    |   |
|  04_init_shards.js             | 21/05/2026 01:11 | JavaScript Source ... |   |
|  05_start_mongos.bat         | 21/05/2026 02:34 | Windows Batch File    |   |
|  06_add_shards.js            | 21/05/2026 03:07 | JavaScript Source ... |   |
|  07_enable_sharding.js       | 21/05/2026 12:11 | JavaScript Source ... |   |
|  08_import_data.bat          | 21/05/2026 13:03 | Windows Batch File    |   |
|  09_check_sharding_status.js | 21/05/2026 13:39 | JavaScript Source ... |   |
|  10_query1_only.js           | 21/05/2026 14:57 | JavaScript Source ... |   |
|  10_query2_only.js           | 21/05/2026 15:01 | JavaScript Source ... |   |
|  10_query3_only.js           | 21/05/2026 15:02 | JavaScript Source ... |   |

## Query1 – what is the average salary and average contract duration

In this first question it asks for the average salary and average duration of working contract for laid off employees in each department. It also only includes employees who had over 70% chance of career progression and over 70% satisfaction rate

In the query below, laid off employees is represented using `left_company: true`, the query also filters employees using the conditions `left_company: true`, `career_progression_score: { $gt: 0.7 }` and `satisfaction_score: { $gt: 0.7 }`

The results then are grouped by department, for each department, the query calculates, average salary and average contract duration in months.

Finally, the query was run the following command, and results were displayed as showed

```
C:\Users\reyco>mongosh --port 27017 C:\mongoClusterProject\scripts\10_query1_only.js
```

```
db.employee_burnout.aggregate([
  {
    $match: {
      left_company: true,
      career_progression_score: { $gt: 0.7 },
      satisfaction_score: { $gt: 0.7 },
      department: { $nin: [null, "nan", ""] }
    }
  },
  {
    $group: {
      _id: "$department",
      average_salary: { $avg: "$salary" },
      average_contract_duration_months: { $avg: "$tenure_months" }
    }
  },
  {
    $project: {
      _id: 0,
      department: "$_id",
      average_salary: { $round: ["$average_salary", 2] },
      average_contract_duration_months: { $round: ["$average_contract_duration_months", 2] }
    }
  },
  {
    $sort: { department: 1 }
  }
]).forEach(printjson);
```

```
Query 1: Average salary and average tenure for laid off em
{
  department: 'Accounting',
  average_salary: 80185.77,
  average_contract_duration_months: 90.55
}
{
  department: 'Accounts Payable',
  average_salary: 74343.5,
  average_contract_duration_months: 93.96
}
{
  department: 'Accounts Receivable',
  average_salary: 81444.66,
  average_contract_duration_months: 70.19
}
{
  department: 'Admin Offices',
  average_salary: 72641.9,
  average_contract_duration_months: 69.8
}
{
  department: 'Analytics',
  average_salary: 86101.76,
  average_contract_duration_months: 81.68
}
```

## Query 2- Department with the most laid off employees

The second question ask which department has the largest number of laid off employees. I placed the asked query inside the script 10\_query2\_only.js , this filters employees where left\_company: true.

It then groups the records by department and counts how many laid off employees are in each department. Finally, the results are sorted in descending order and the top result is returned.

Query was run using

```
C:\Users\reyco>mongosh --port 27017 C:\mongoClusterProject\scripts\10_query2_only.js
```

And here is there returned department with the highest number of laid off employees

```
Query 2: Department with the large
{
  laid_off_count: 43998,
  department: 'Sales & Marketing'
}
```

```
db.employee_burnout.aggregate([
  {
    $match: {
      left_company: true,
      department: { $nin: [null, "nan", ""] }
    }
  },
  {
    $group: {
      _id: "$department",
      laid_off_count: { $sum: 1 }
    }
  },
  {
    $sort: { laid_off_count: -1 }
  },
  {
    $limit: 1
  },
  {
    $project: {
      _id: 0,
      department: "$_id",
      laid_off_count: 1
    }
  }
]).forEach(printjson);
```

**Query 3** – Teams with high sentiment, high meeting frequency, resting stress and low burnout risk

The third and final question of this first part ask for all teams in every department that have, high sentiment score above 60%, high meeting frequency above 70%, resting stress level and low burnout risk. The project defines resting stress as 0% – 25% and low burnout risk as 0% – 49%

For this question I placed the query inside 10\_query3\_only.js, also the dataset does not contain a direct field called team, so the role field was used to represent employee teams and groups

The query filters record using, team\_sentiment: { \$gt: 0.6 }, meeting\_participation: { \$gt: 0.7 }, stress\_level: { \$lte: 0.25 } and burnout\_risk: { \$lte: 0.49 }

The results are then grouped by department role and finally ran using

```
C:\Users\reyco>mongosh --port 27017 C:\mongoClusterProject\scripts\10_query3_only.js
```

Finally, the output shows each matching department and the role used as the team value

```
{
  department: 'Finance',
  team: ' Shop Assistant'
}
{
  department: 'Finance',
  team: ' Software Developer'
}
{
  department: 'Finance',
  team: ' Software Development Engineer In Test (SDET)'
}
{
  department: 'Finance',
  team: ' Software Engineer'
}
{
  department: 'Finance',
  team: ' Software Engineer II'
}
{
  department: 'Finance',
  team: ' Solutions Architect'
}
```

```
db.employee_burnout.aggregate([
  {
    $match: {
      team_sentiment: { $gt: 0.6 },
      meeting_participation: { $gt: 0.7 },
      stress_level: { $lte: 0.25 },
      burnout_risk: { $lte: 0.49 },
      department: { $nin: [null, "nan", ""] },
      role: { $nin: [null, "nan", ""] }
    }
  },
  {
    $group: {
      _id: {
        department: "$department",
        team: "$role"
      }
    }
  },
  {
    $project: {
      _id: 0,
      department: "$_id.department",
      team: "$_id.team"
    }
  },
  {
    $sort: {
      department: 1,
      team: 1
    }
  }
]).forEach(printjson);
```



## Part 2 Atlas Cluster

### Setup